UniRide (CarpoolAppFrontBack)

Security Threat Model Report
STRIDE Methodology

Prepared by: Hayyan Khan
Date: February 28, 2026
Stack: ASP.NET Core .NET 9 · React 19 (Vite) · SQLite (EF Core) · SignalR

Task 1 — System Definition

Overview

UniRide is a university carpooling web application for Habib University students (email domain @st.habib.edu.pk). Drivers post rides; passengers search and book them. The backend is ASP.NET Core (.NET 9) with a SQLite database (EF Core), JWT authentication (HS256), SignalR-based chat, and email OTP registration via Gmail SMTP (MailKit). The frontend is a React 19 SPA (Vite) using axios and @microsoft/signalr.

Components

Component	Technology / File	Role
React SPA	Vite + React 19, axios, @microsoft/signalr	LoginPage, RegisterPage, DriverDashboard, PassengerDashboard, CreateRidePage, PassengerProfile, ChatPage
ASP.NET Core API	Program.cs	JWT middleware, CORS (localhost:58562 only), SignalR hub at /hubs/chat, EF Core SQLite
AuthController	Controllers/Shared/AuthController.cs	OTP send/verify, register, login — JWT issued here; OTP stored in static ConcurrentDictionary
Driver Controllers	Controllers/Driver/	DriverDashboard, RideManagement, RideRequest, DriverProfile — all [Authorize(Roles="driver")]
Passenger Controllers	Controllers/Passenger/	Booking, PassengerDashboard, PassengerProfile, RideSearch — all [Authorize(Roles="passenger")]
MessageController	Controllers/Shared/MessageController.cs	Get/send messages per ride — [Authorize]; checks ConversationMembers before access
ChatHub	Hub/ChatHub.cs	SignalR at /hubs/chat — JoinRideGroup, LeaveRideGroup, SendMessage — NO [Authorize]
EmailService	EmailService/EmailService.cs	Sends OTPs via MailKit, smtp.gmail.com:587, STARTTLS
CarpoolDbContext	Data/CarpoolDbContext.cs	10 tables; EF Core; unique indexes on UniversityEmail and NumberPlate
SQLite DB	carpoolapp.db (committed to repo)	All persistent data — the actual .db file is tracked in git
appsettings.json	appsettings.json (committed to repo)	JWT key, Gmail credentials, DB connection string — all in plaintext

User Roles

Role	How Assigned	Access
Driver	Client sends role="driver" in login body; server creates Driver record; JWT claim = "driver"	Create rides, manage vehicles, accept/reject requests, chat
Passenger	Client sends role="passenger" in login body; server creates Passenger record; JWT claim = "passenger"	Browse/search rides, request rides, view accepted rides, chat
Admin	Does not exist in the codebase	N/A — no admin controller, role, or interface

Trust Boundaries

Boundary	Separates	Enforcement
TB1 — Public/Browser	Untrusted browser from server	UseHttpsRedirection; CORS restricted to localhost:58562
TB2 — Frontend/API	React SPA from controllers	JWT Bearer middleware; Vite proxy for /api and /hubs
TB3 — Backend/Data	Controllers from SQLite and config	EF Core LINQ only; appsettings.json on server filesystem

Architecture Diagram

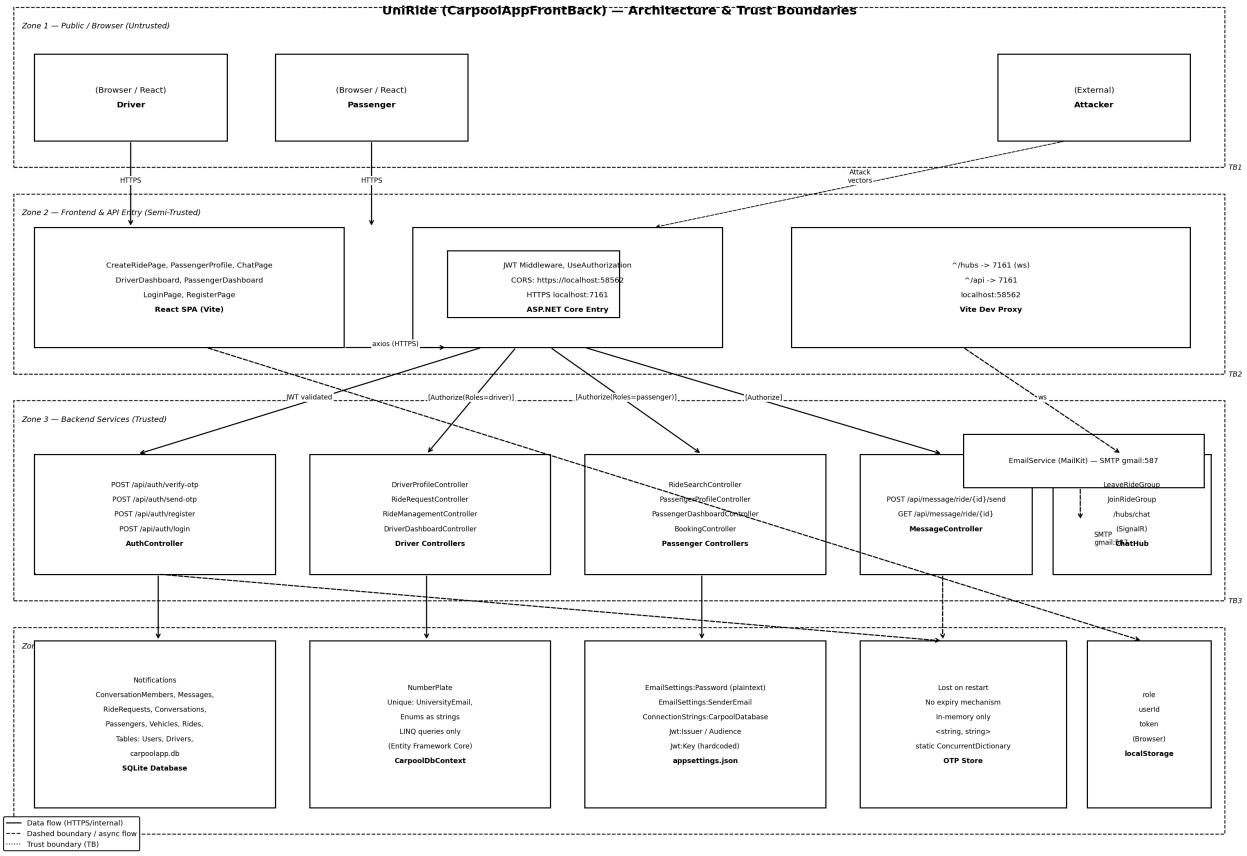

Figure 1 — UniRide Architecture with Trust Boundaries

Task 2 — Asset Identification

ID	Asset	Location in Code	C	I	A	Ac
A1	JWT Signing Key	appsettings.json Jwt:Key — hardcoded default string	Crit	Crit	Med	Med
A2	Gmail SMTP Credentials	appsettings.json EmailSettings — plaintext	Crit	Med	Med	Med
A3	Password Hashes	Users table PasswordHash (PBKDF2/ASP.NET Identity)	Crit	Crit	Med	Crit
A4	JWT Bearer Tokens	Browser localStorage key "token"	Crit	Crit	Med	Crit
A5	Student PII	Users table — FullName, UniversityEmail, PhoneNumber	Crit	Med	Med	Crit
A6	OTP Codes	Static ConcurrentDictionary in AuthController	Crit	Crit	Med	Med
A7	Ride Data	Rides and RideRequests tables	Med	Crit	Crit	Med
A8	Chat Messages	Messages table; SignalR broadcasts	Med	Med	Med	Med
A9	Vehicle Records	Vehicles table — NumberPlate unique index	Low	Med	Med	Low
A10	SQLite DB File	carpoolapp.db committed to git repository	Crit	Crit	Crit	Crit
A11	Role Claim in JWT	AuthController.Login() — taken directly from dto.Role	Crit	Crit	Med	Crit
A12	Conversation Membership	ConversationMembers table	Med	Crit	Med	Med

C = Confidentiality, I = Integrity, A = Availability, Ac = Accountability | Crit = Critical, Med = Medium

Task 3 — STRIDE Threat Model

S=Spoofing · T=Tampering · R=Repudiation · I=Information Disclosure · D=Denial of Service · E=Elevation of Privilege

ID	S	Threat Name	Source in Code	Description	Risk
TH-01	**S**	**Client-Controlled Role at Login**	`AuthController.cs Login(): role taken from dto.Role, used as JWT claim`	Any user can claim any role (driver/passenger) by changing the login request body. No server-side validation of permitted role.	**High**
TH-02	**I**	**Hardcoded JWT Signing Key**	`appsettings.json Jwt:Key = "YourSuperSecretKey123456789101112"`	Default key committed to the repository. Anyone with repo access can forge valid JWTs for any user and any role.	**High**
TH-03	**I**	**Gmail Credentials in Config File**	`appsettings.json EmailSettings:Password = "jbqa diyg lusd cjju"`	Real Gmail app password committed in plaintext. Enables email abuse and interception of OTP codes.	**High**
TH-04	**I**	**JWT Stored in localStorage**	`LoginPage.jsx: localStorage.setItem("token", res.data.token)`	localStorage is readable by any JavaScript on the page. XSS anywhere in the app yields the active session token.	**High**
TH-05	**S**	**OTP Generated with System.Random**	`AuthController.cs SendOtp(): new Random().Next(100000, 999999)`	System.Random is not cryptographically secure. 6-digit space (900,000 values) is brute-forceable, especially without rate limiting.	**Medium**
TH-06	**D**	**No Rate Limiting on Auth Endpoints**	`Program.cs: no AddRateLimiter(); no attempt counters in AuthController`	Unlimited automated requests accepted on /api/auth/login, /api/auth/send-otp, /api/auth/verify-otp. Enables brute-force attacks.	**High**
TH-07	**R**	**OTP Has No Server-Side Expiry**	`AuthController.cs OtpStore: no timestamp stored, no cleanup task`	OTP entries persist until server restart. Frontend timer is decorative; server never rejects an old OTP.	**Medium**
TH-08	**E**	**No Ownership Check on GetRideLocations**	`BookingController.cs GetRideLocations(int rideId): no passenger membership check`	Any authenticated passenger can retrieve location data for any rideId, regardless of whether they have a ride request.	**Medium**
TH-09	**T**	**Duplicate request-ride Endpoint Without Guard**	`PassengerDashboardController.cs POST request-ride: no duplicate-request check`	A second request-ride endpoint in PassengerDashboardController does not check for existing requests, allowing duplicate submissions.	**Medium**
TH-10	**E**	**ChatHub Has No [Authorize]**	`Hub/ChatHub.cs: class ChatHub : Hub — no [Authorize] attribute`	The SignalR hub at /hubs/chat is accessible without a JWT. Unauthenticated clients can join any ride group and receive messages.	**High**
TH-11	**E**	**JoinRideGroup Has No Membership Check**	`Hub/ChatHub.cs JoinRideGroup(): no ConversationMembers DB lookup`	Even an authenticated user who is not part of a ride can call JoinRideGroup and receive all real-time chat messages for that ride.	**High**
TH-12	**I**	**SQLite DB File Committed to Repo**	`carpoolapp.db + .db-shm + .db-wal committed and tracked by git`	The live database file (all users, rides, messages) is in the repository. Anyone who has cloned the repo has a full copy.	**High**
TH-13	**I**	**No Encryption at Rest for Database**	`Program.cs: options.UseSqlite("Data Source=carpoolapp.db") — no SQLCipher`	The SQLite file is unencrypted. Filesystem or repo access yields all data with no decryption step.	**Medium**

ID	S	Threat Name	Source in Code	Description	Risk
TH-14	**T**	**Email Domain Validation Commented Out**	`Models/User.cs: [RegularExpression] attribute commented out; no domain check in Register()`	Only client-side JS enforces @st.habib.edu.pk. A direct POST to /api/auth/register registers any email domain.	**Medium**
TH-15	**I**	**Exception Details in API Error Responses**	`PassengerDashboardController.cs, PassengerProfileController.cs: error = ex.Message in 500 responses`	Raw .NET exception messages returned to the client reveal internal implementation details.	**Medium**
TH-16	**R**	**No Security Event Logging**	`Program.cs: default log levels only; no ILogger in any controller`	Auth failures, IDOR attempts, and OTP brute-force events leave no server-side trace. Incident detection is impossible.	**High**
TH-17	**I**	**All Secrets Committed to Repository**	`appsettings.json (JWT key, Gmail password) + carpoolapp.db in git history`	All application secrets are permanently stored in version history. Any past clone holds a copy.	**High**

Risk Level	Count	Threat IDs
High	9	TH-01, TH-02, TH-03, TH-04, TH-06, TH-10, TH-11, TH-12, TH-16, TH-17
Medium	7	TH-05, TH-07, TH-08, TH-09, TH-13, TH-14, TH-15
Low	1	—

Task 4 — Secure Architecture Design

Control	Architectural Change	Threats Mitigated
Determine role from database, not client input	Remove Role field from LoginDto. Query Drivers/Passengers tables at login to determine role.	TH-01
Replace hardcoded JWT key with env variable	Generate a random 256-bit key. Store in environment variable (ASPNETCORE_JWT_KEY). Use .NET User Secrets in development.	TH-02, TH-17
Move secrets out of appsettings.json	Use environment variables or .NET User Secrets for JWT key, Gmail password, and DB connection string. Add appsettings.json to .gitignore for sensitive values.	TH-03, TH-17
Store JWT in HttpOnly cookie	Set-Cookie with HttpOnly, Secure, SameSite=Strict. Remove localStorage.setItem("token") from frontend.	TH-04
Replace System.Random OTP with RandomNumberGenerator	Use System.Security.Cryptography.RandomNumberGenerator.GetInt32(100000,1000000). Store OTP in DB with IssuedAt timestamp. Reject entries older than 5 minutes.	TH-05, TH-07
Add rate limiting middleware	AddRateLimiter() in Program.cs. Fixed-window policy on /api/auth/* endpoints (e.g. 5 req/10 min per email).	TH-06
Add [Authorize] to ChatHub; check membership in JoinRideGroup	Apply [Authorize] to ChatHub class. In JoinRideGroup, query ConversationMembers table to verify the calling user belongs to that ride before AddToGroupAsync.	TH-10, TH-11
Remove SQLite DB file from repository	Add *.db, *.db-shm, *.db-wal to .gitignore. Purge from git history using git filter-repo. Treat all committed data as compromised.	TH-12
Encrypt database at rest	Use SQLCipher with Microsoft.EntityFrameworkCore.Sqlite. Store encryption key in environment variable.	TH-13
Re-enable server-side email domain validation	Uncomment [RegularExpression] in User.cs, or add explicit domain check in AuthController.Register().	TH-14
Add ownership check to GetRideLocations	Before returning location data in BookingController, verify the passenger has a RideRequest for that rideId.	TH-08
Remove exception details from error responses	Replace ex.Message in 500 responses with a generic string. Log full exception server-side using ILogger.	TH-15
Add structured security logging	Inject ILogger in all controllers. Log auth successes/failures, OTP events, and authorisation denials.	TH-16
Consolidate duplicate request-ride endpoint	Remove POST /api/passengerdashboard/request-ride. Keep only the BookingController version, which already has a duplicate-request guard.	TH-09

Task 5 — Risk Treatment

ID	Risk	Treatment	Justification
TH-01	High	Mitigate	Server-side role determination removes direct privilege escalation.
TH-02	High	Mitigate	Key is publicly known; must be rotated immediately and stored outside source.
TH-03	High	Mitigate	Gmail password is live and functional; rotate and remove from config immediately.
TH-04	High	Mitigate	HttpOnly cookie is a one-change fix that eliminates JS-accessible token storage.
TH-05	Medium	Mitigate	One-line swap to RandomNumberGenerator; significant security improvement.
TH-06	High	Mitigate	Built-in ASP.NET Core rate limiting (AddRateLimiter) requires no extra dependency.
TH-07	Medium	Mitigate	DB-persisted OTP with IssuedAt column; same change as TH-16 logging fix.
TH-08	Medium	Mitigate	Single additional DB query in GetRideLocations.
TH-09	Medium	Mitigate	Remove duplicate endpoint; duplicate guard already exists in BookingController.
TH-10	High	Mitigate	Adding [Authorize] to ChatHub is a one-line change.
TH-11	High	Mitigate	Add ConversationMembers lookup in JoinRideGroup before AddToGroupAsync.
TH-12	High	Mitigate	Remove DB from repo and purge git history; committed data must be treated as compromised.
TH-13	Medium	Mitigate	SQLCipher integration prevents plaintext DB access at rest.
TH-14	Medium	Mitigate	Uncomment one attribute or add one line in Register().
TH-15	Medium	Mitigate	Replace ex.Message with generic string; log server-side.
TH-16	High	Mitigate	ILogger is already part of the ASP.NET Core framework; no new dependency.
TH-17	High	Mitigate	Rotate all secrets; remove from repo; purge git history.

Residual Risk

Residual Risk	Reason
Git history permanently exposes the committed JWT key and Gmail password	Any clone before purge retains the secrets. All JWT tokens signed with the old key are permanently compromised.
SQLite has no row-level access control	Server compromise yields full unrestricted DB access regardless of application-layer controls.
No MFA at login	OTP is used only at registration. Credential theft leads directly to account takeover.
Chat messages not end-to-end encrypted	Messages are stored in plaintext in the Messages table; readable by anyone with DB access.

Task 6 — Final Report

System Summary

UniRide is a university carpooling platform for Habib University students. The backend is ASP.NET Core (.NET 9) with 10 REST controllers, SignalR chat, and Gmail OTP registration. SQLite (EF Core) is the data store. The frontend is React 19 (Vite). Authentication uses JWT HS256 tokens (6-hour expiry) stored in browser localStorage. Role (driver/passenger) is supplied by the client at login. No admin role or interface exists in the codebase.

Architecture Diagram

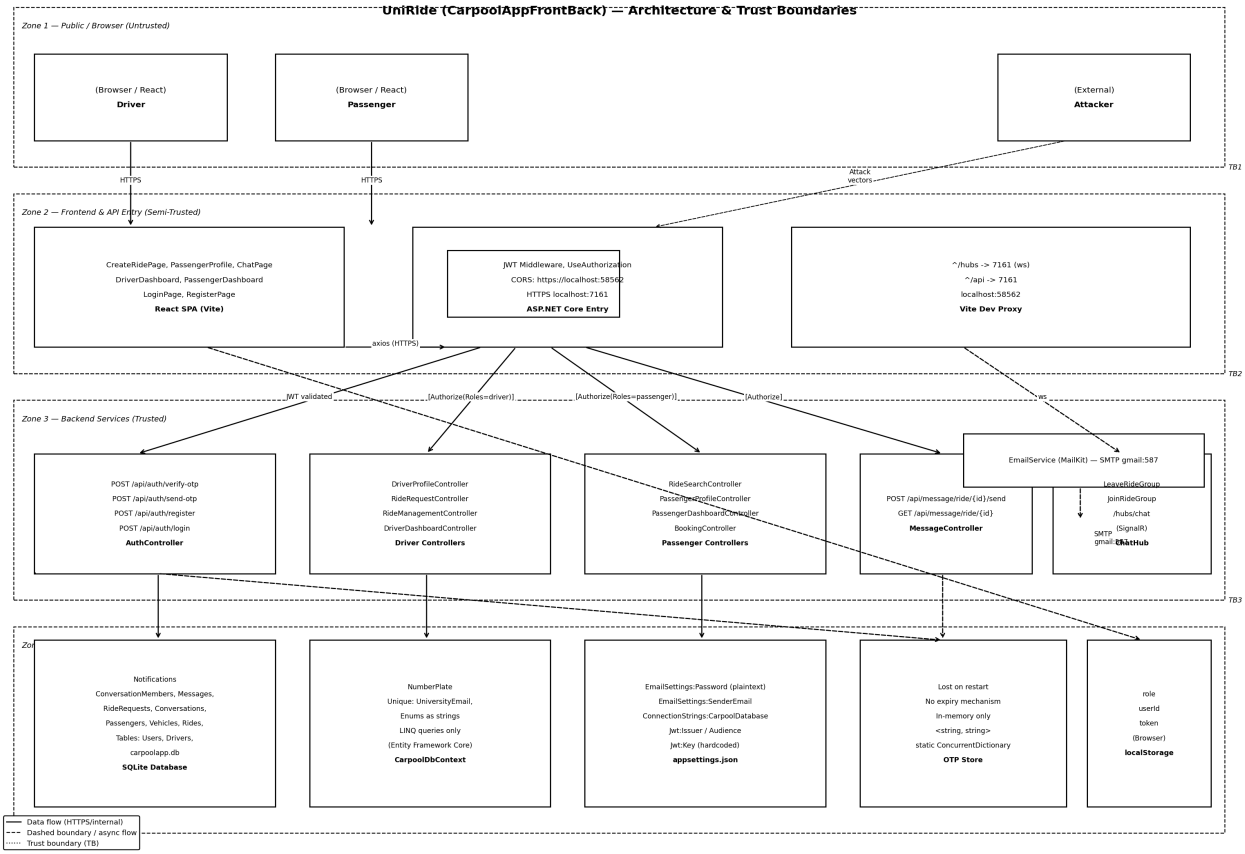

Figure 2 — UniRide Architecture and Trust Boundaries

Summary Tables

Metric	Value
Total threats identified	17
High risk	9 — TH-01,02,03,04,06,10,11,12,16,17
Medium risk	7 — TH-05,07,08,09,13,14,15
Low risk	1
Treatment decisions	16 Mitigate · 1 Accept (TH — none explicitly accepted; TH-15 borderline Accept)
Residual risks	4 (git history, no row-level DB control, no MFA at login, plaintext chat storage)

Priority Actions

- **TH-02/03/17** — Rotate the JWT key and Gmail password immediately. Remove appsettings.json secrets and the carpoolapp.db file from the repository and purge git history.
- **TH-10/11** — Add [Authorize] to ChatHub. Add ConversationMembers lookup in JoinRideGroup.
- **TH-01** — Fix role assignment: query the DB at login, do not trust the client-supplied role field.

- **TH-04** — Move JWT from localStorage to HttpOnly, Secure, SameSite=Strict cookie.
- **TH-06** — Add AddRateLimiter() to Program.cs with limits on all /api/auth/* endpoints.
- **TH-16** — Add ILogger to all controllers; log auth and OTP events.

Assumptions

- The application is in a development environment on localhost. No production deployment configuration was reviewed.
- The committed carpoolapp.db file is treated as containing live data, evidenced by active WAL and SHM files being present.
- No payment processing, OAuth, or SSO is present. Gmail SMTP via MailKit is the only external service.
- No dynamic testing or runtime exploitation was performed. This is a code-level security review.